



TECHNICAL OVERVIEW

GPS Ushering and Events — How It's Built

Architecture, stack, and project layout — for a developer picking this up cold.

1. Stack

LAYER	CHOICE	WHY
Language / framework	Python 3 + FastAPI	Requested — the maintainer is Python/FastAPI-fluent, not JS
Templating	Jinja2 (via Starlette's <code>Jinja2Templates</code>)	Ships with FastAPI; real template inheritance (<code>{% extends %}</code> / <code>{% block %}</code>)
Content storage	Markdown + YAML front matter, read with <code>python-frontmatter</code>	Git-native, diffable, and directly editable by the CMS below
Content editor	Decap CMS (static JS app, CDN-loaded)	Zero-backend git-based CMS; commits straight to this repo
Database	SQLAlchemy 2.x ORM — SQLite locally, Postgres in production	One booking table; no need for anything heavier
Auth (booking admin)	Single hardcoded admin user (env vars) + signed cookie session (<code>itsdangerous</code>)	One internal user (the business owner) — no need for a user table
Server	uvicorn (ASGI)	Standard FastAPI runtime
Client-side JS	One vanilla file, <code>app/static/js/main.js</code>	Mobile nav, theme toggle, FAQ accordion, gallery filter/lightbox, booking form fetch — no framework, no build step

There is no Node.js, npm, or build step anywhere in this project. Everything — pages, the booking API, and both admin tools — is one FastAPI app, run with `uvicorn app.main:app`.

Two distinct "admin" surfaces — don't conflate them

PATH	WHAT IT IS	AUTH
<code>/admin</code>	Booking inquiries dashboard (this app's own Jinja2 templates)	Single admin user, signed cookie (<code>app/security.py</code>)
<code>/content-admin</code>	Decap CMS — edits <code>content/*.md</code> and <code>content/settings.json</code> via git commits	GitHub OAuth, proxied through <code>app/routers/oauth.py</code>

2. Project Structure

```
.
├── app/
│   ├── main.py          # FastAPI app assembly: static mounts, routers, DB init
│   ├── config.py        # pydantic-settings — reads env vars / .env
│   ├── database.py      # SQLAlchemy engine/session (sqlite:// or postgresql+psycopg://)
│   ├── models.py        # Booking ORM model + BookingStatus enum
│   ├── schemas.py       # Pydantic request/response schemas
│   ├── security.py      # Signed-cookie session helpers for /admin
│   ├── content.py       # Reads content/*.md + settings.json -> plain dicts
│   └── routers/
│       ├── pages.py     # Every public page (/ , /about.html, ... /book-us.html)
│       ├── bookings.py  # POST /api/bookings (public, honeypot-filtered)
│       ├── admin.py     # /admin login + booking dashboard + status updates
│       └── oauth.py     # GitHub OAuth proxy for the content-admin CMS
│   └── templates/
│       ├── base.html    # Layout: head, topbar, navbar, footer, WhatsApp float
│       ├── partials/    # topbar.html, navbar.html, footer.html
│       ├── pages/       # index.html, about.html, services.html, ...
│       └── admin/       # login.html, dashboard.html (booking admin only)
│   └── static/
│       ├── css/style.css # All styling — CSS vars, light/dark theme tokens
│       ├── js/main.js   # All client-side interactivity
│       └── admin.css    # Booking-dashboard-only styling
├── content/             # The CMS-editable data, read by app/content.py
│   ├── settings.json    # Site-wide: contact info, social links, site name
│   ├── services/*.md    # 9 entries — title, icon, description, highlights
│   ├── testimonials/*.md # name, role, rating, quote (markdown body)
│   ├── gallery/*.md     # label, category, image path
│   └── faq/*.md         # question (front matter) + answer (markdown body)
├── content-admin/       # Decap CMS: static index.html + config.yml
├── images/uploads/      # Photos uploaded through the content-admin land here
├── docs/                # Non-technical project docs (this file's siblings)
├── requirements.txt
├── Procfile             # `web: uvicorn app.main:app --host 0.0.0.0 --port $PORT`
└── .env.example
```

3. How a Page Request Actually Resolves

1. A route in `app/routers/pages.py` (e.g. `GET /services.html`) calls `content.get_services()`, which globs `content/services/*.md`, parses each with `python-frontmatter`, converts the Markdown body to HTML (`body_html`), and sorts by an `order` field.
2. The route builds a context dict via a shared `base_context(request, **extra)` helper (also in `pages.py`), which always injects `settings` (from `content/settings.json`) and `footer_services` (first 4 services, for the footer's link list).
3. `templates.TemplateResponse(request, "pages/services.html", context)` renders that template, which does `{% extends "base.html" %}` and fills in `{% block content %}`.
4. `base.html` pulls in the shared `partials/topbar.html`, `navbar.html`, `footer.html`, all reading from the same `settings` context variable.

Nothing is pre-built or cached to disk — every request re-reads the content files fresh. For a site this size (a few dozen small Markdown files) this costs single-digit milliseconds and means a content-admin save is live on the very next request, no rebuild step, no cache to bust.

content-admin (Decap CMS) auth flow

`content-admin/config.yml` configures Decap's github backend pointing `base_url` at this same FastAPI service. The flow: Decap opens a popup to `/auth` (redirects to GitHub's OAuth authorize page) → GitHub redirects back to `/callback?code=...` → `app/routers/oauth.py` exchanges the code server-side (holding the OAuth app's client secret) → returns a small HTML page that `postMessages` the token back to the CMS popup's opener, per Decap's documented external-OAuth-provider handshake.

Requires a GitHub OAuth App (github.com/settings/developers) with its callback URL set to `<deployed-url>/callback`, and `GITHUB_OAUTH_CLIENT_ID/GITHUB_OAUTH_CLIENT_SECRET` set in the environment. Not yet configured — `content-admin/config.yml`'s `base_url` is still a placeholder.

Booking Admin auth

Single admin user, credentials from `ADMIN_USERNAME/ADMIN_PASSWORD` env vars, compared with `hmac.compare_digest`. On success, `app/security.py` issues an `itsdangerous`-signed cookie (no server-side session store — the cookie itself is the session, verified by signature + a 12-hour max age on every protected request). `COOKIE_SECURE` must be `false` for local `http://` testing (browsers silently drop a Secure cookie over plain HTTP) and should stay `true` in production.

4. Data Model

One table: bookings (app/models.py) — name, phone, email, event_type, event_date, guest_count, location, message, a status enum (new/contacted/confirmed/completed/cancelled), and created_at. POST /api/bookings (app/routers/bookings.py) is public and unauthenticated by design — it's the public booking form's submit endpoint — but drops anything where the bot_field honeypot is filled in without storing or acknowledging it any differently to a bot.

app/database.py rewrites a Railway-style postgres:// URL to postgresql+psycopg:// (psycopg 3) automatically; locally it defaults to sqlite:///./bookings.db with no setup required.

5. Running It Locally

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
ADMIN_USERNAME=admin ADMIN_PASSWORD=devpass SECRET_KEY=dev COOKIE_SECURE=false \
uvicorn app.main:app --reload --port 8000
```

Then `http://localhost:8000/` for the site, `/admin` (login: admin/devpass) for bookings, `/content-admin` to preview the CMS shell (login won't work until the GitHub OAuth app is set up).

6. Deployment Plan

Railway, one service, running the Procfile's `uvicorn app.main:app --host 0.0.0.0 --port $PORT`. Add a Postgres plugin (Railway injects `DATABASE_URL` automatically). Set the env vars from `.env`, example: `SECRET_KEY`, `ADMIN_USERNAME/ADMIN_PASSWORD`, `COOKIE_SECURE=true`, `GITHUB_OAUTH_CLIENT_ID/SECRET`. **Cloudflare** is DNS-only in front of it (or skip it and point the domain straight at Railway).

7. Known Gaps / Worth Revisiting

- No automated tests yet — everything so far has been verified manually (curl + Playwright screenshots) rather than with a test suite.
- No pagination on the booking dashboard — fine at dozens of bookings, will want it eventually.
- No rate limiting on POST /api/bookings beyond the honeypot field.
- Single hardcoded admin user — fine for one business owner, would need real user accounts to support more than one person.
- `content-admin`'s GitHub OAuth app and Railway URL are both still placeholders (`REPLACE-WITH-...` in `content-admin/config.yml` and nothing in `.env` yet).